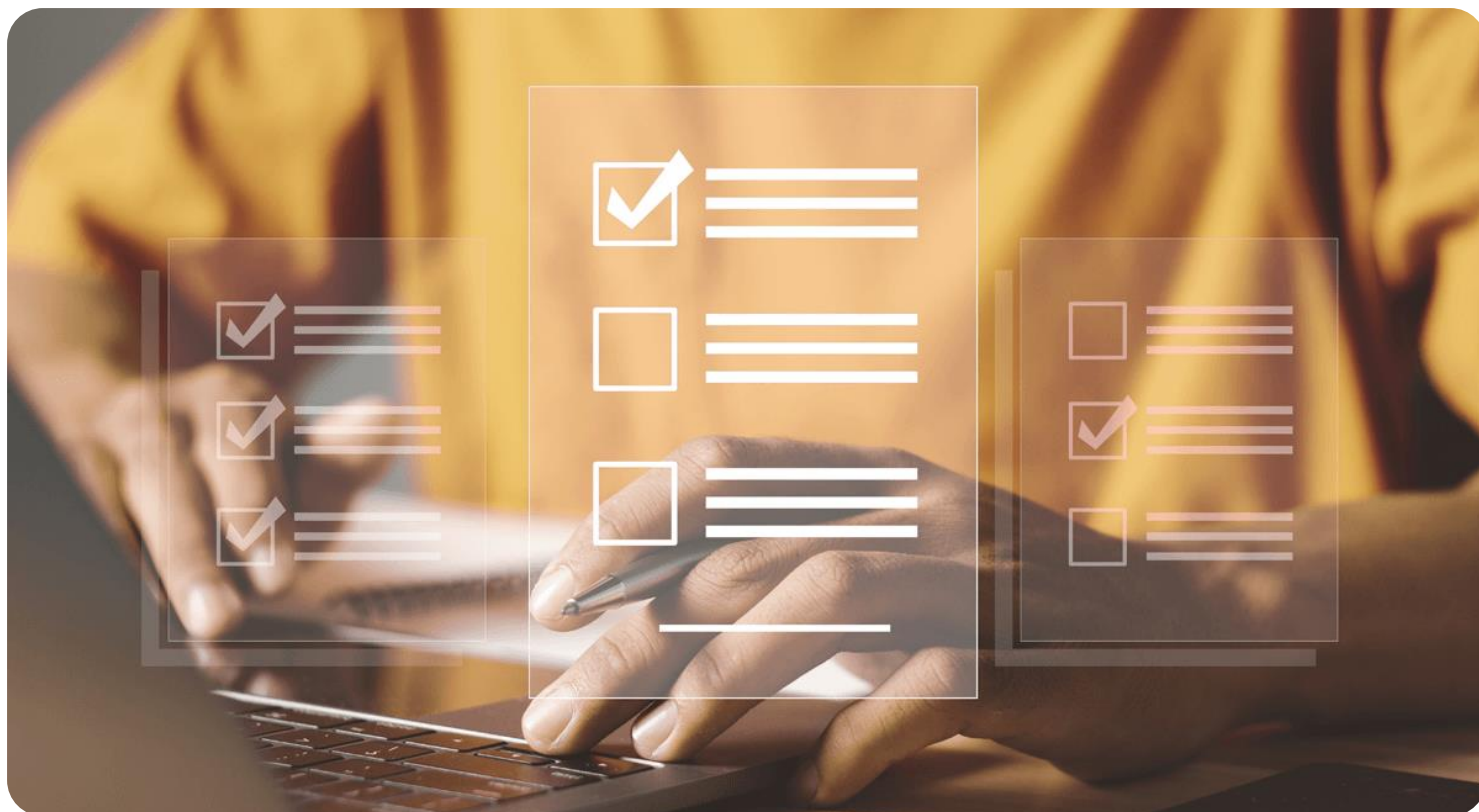




Semana 3

Desarrollo Backend III (PBY2203)

Nombre estudiante:	Jorge Germán Gallardo Heck
Carrera: Analista Programado Computacional	
Profesor: Alonso Esteban Castillo Pizarro	Fecha: 31/08/26

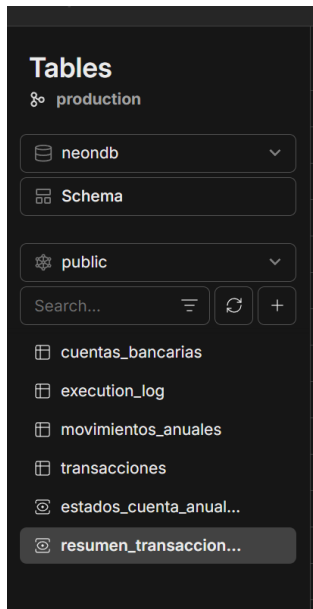
Informe Técnico — Migración de Procesos Batch Legacy (Banco XYZ)

Base de datos:

PostgreSQL alojado en Neon elegido por buen soporte con Spring Batch, tier gratuito con persistencia permanente.

Persistencia:

JDBC puro (no JPA/Hibernate) — más liviano y predecible para batch processing masivo. El esquema se define a mano en `schema-postgresql.sql`: 3 tablas de datos (`transacciones`, `cuentas_bancarias`, `movimientos_anuales`), la tabla `execution_log` para trazabilidad, y 2 vistas de resumen (`resumen_transacciones_diarias`, `estados_cuenta_anuales`).



Arquitectura:

3 Jobs independientes (uno por proceso legacy), cada uno con su propio BatchConfig, disparados en secuencia por un CommandLineRunner con JobOperator.

CommandLineRunner:

```
19  */
20  @Configuration
21  public class JobLauncherConfig {
22
23      @Bean
24      public CommandLineRunner lanzarJobs(JobOperator jobOperator,
25                                          Job transaccionJob,
26                                          Job cuentaBancariaJob,
27                                          Job movimientoAnualJob) {
28          return String[] args -> {
29              var parametrosTransaccion = new JobParametersBuilder()
30                  .addLong( name: "timestamp", System.currentTimeMillis())
31                  .toJobParameters();
32              jobOperator.start(transaccionJob, parametrosTransaccion);
33
34              var parametrosCuenta = new JobParametersBuilder()
35                  .addLong( name: "timestamp", System.currentTimeMillis())
36                  .toJobParameters();
37              jobOperator.start(cuentaBancariaJob, parametrosCuenta);
38
39              var parametrosMovimiento = new JobParametersBuilder()
40                  .addLong( name: "timestamp", System.currentTimeMillis())
41                  .toJobParameters();
42              jobOperator.start(movimientoAnualJob, parametrosMovimiento);
43
44              System.exit( status: 0);
45          };
46      }
47  }
```

Manejo de errores:

Políticas de skip y retry personalizadas (BancoXyzSkipPolicy, BancoXyzRetryPolicy) — los registros inválidos se descartan sin detener el Job, mientras que errores transitorios (no de negocio) se reintentan hasta 3 veces antes de fallar.

```
@Slf4j 2 usages
public class BancoXyzRetryPolicy extends SimpleRetryPolicy {

    private static final int MAX_ATTEMPTS = 3; 2 usages
```

Tamaño de chunk:

Política de completion personalizada (BancoXyzCompletionPolicy) — cierra cada chunk por cantidad de ítems o por tiempo transcurrido, lo que ocurra primero, en vez de un tamaño fijo. La cantidad de ítems se ajustó según el volumen del dataset (5 en semana_2, 500 en semana_3 tras la comparación de rendimiento).

```

*/
public class BancoXyzCompletionPolicy extends CompletionPolicySupport { 6 us

    private static final int MAX_ITEMS = 500; 1 usage
    private static final long MAX_TIEMPO_MS = 2000; 1 usage

```

Trazabilidad:

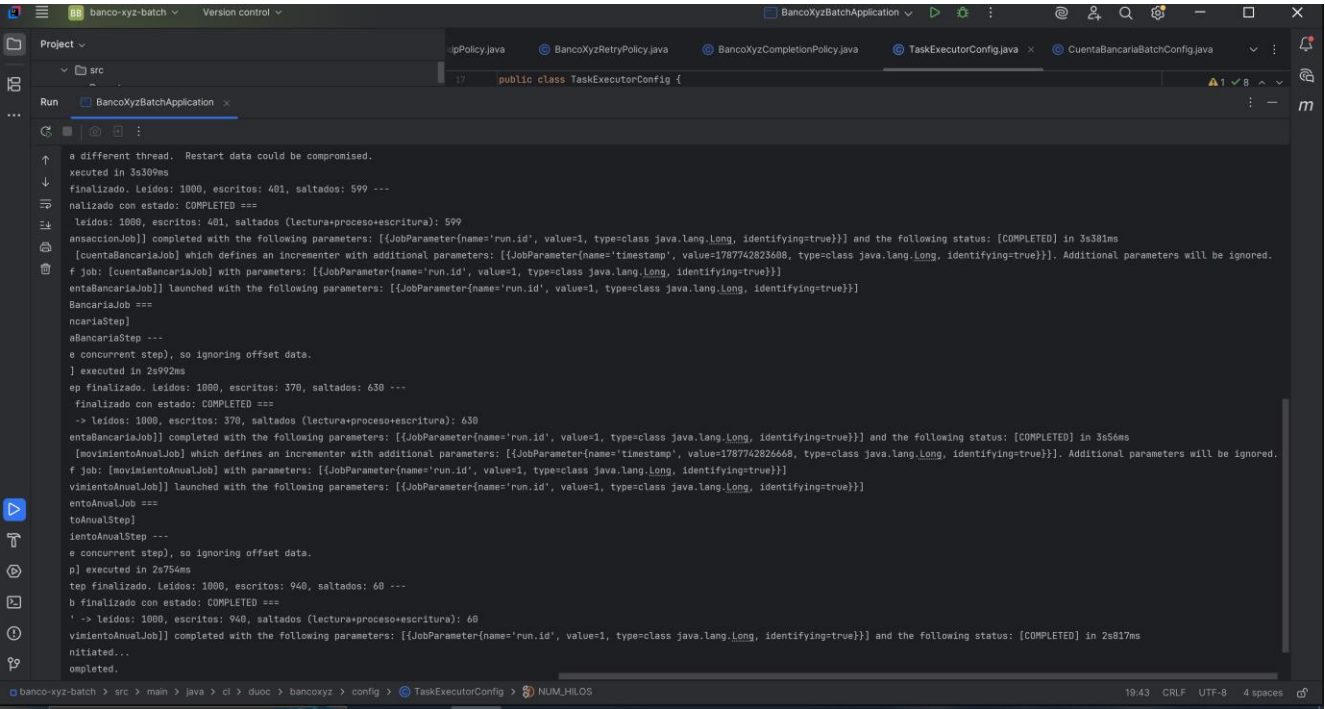
Listeners de Job, Step y Skip en cada uno de los 3 procesos — registran en el log el inicio/fin de cada ejecución, los contadores de lectura/escritura, y el detalle de cada registro saltado. El resumen de cada Step se persiste además en una tabla propia, `execution_log`, ya que Spring Batch 6 no guarda sus tablas de metadata por defecto.

Procesamiento paralelo:

Cada Step se configuró con un TaskExecutor (ThreadPoolTaskExecutor) para procesar varios chunks en paralelo dentro de un mismo Job, con una política de escalamiento fija: **6 hilos** (`corePoolSize = maxPoolSize = 6`, sin escalamiento dinámico), una cola de espera de **100 tareas** para los chunks que excedan la capacidad de los hilos disponibles, y `CallerRunsPolicy` como salvaguarda ante una eventual saturación del pool y la cola. El `FlatFileItemReader` de cada Job se envuelve en un `SynchronizedItemStreamReader`, ya que el Reader original no es seguro para ser accedido por múltiples hilos simultáneamente.

Hilos	Chunk	Tiempo Total(s)
3	50	9.254
6	200	4.691
10	200	4.344
8	200	4.117
8	300	3.152
3	250	3.097
8	550	2.802
3	500	2.884
6	500	2.781

Hilos: 3 Chunk: 50



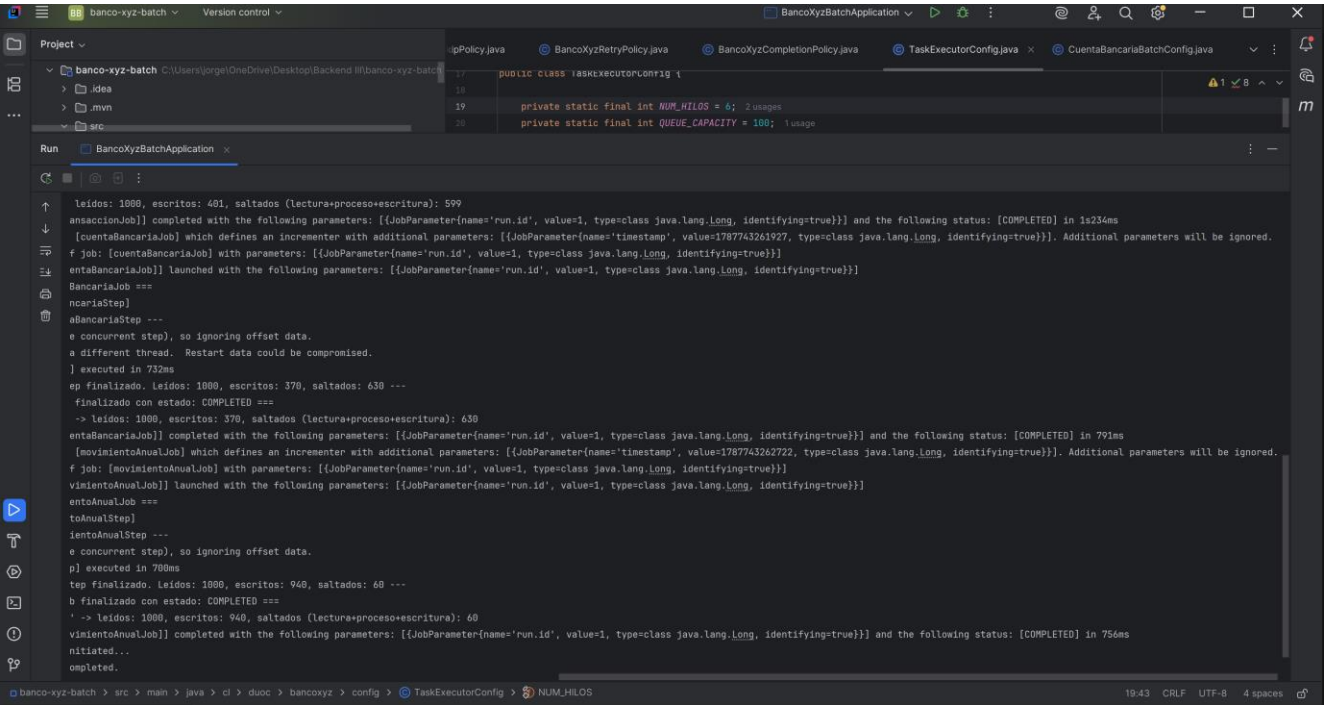
Job: [SimpleJob: [name=transaccionJob]] completed with the following parameters: [{JobParameter{name='run.id', value=1, type=class java.lang.Long, identifying=true}}] and the following status: [COMPLETED] in 3s381ms

Job: [SimpleJob: [name=cuentaBancariaJob]] completed with the following parameters: [{JobParameter{name='run.id', value=1, type=class java.lang.Long, identifying=true}}] and the following status: [COMPLETED] in 3s56ms

Job: [SimpleJob: [name=movimientoAnualJob]] completed with the following parameters: [{JobParameter{name='run.id', value=1, type=class java.lang.Long, identifying=true}}] and the following status: [COMPLETED] in 2s817ms

Hilos	Chunk	Tiempo Total (s)
3	50	9.254

Hilos: 6 Chunk: 500



Job: [SimpleJob: [name=movimientoAnualJob]] completed with the following parameters: [{JobParameter{name='run.id', value=1, type=class java.lang.Long, identifying=true}}] and the following status: [COMPLETED] in 756ms

Job: [SimpleJob: [name=cuentaBancariaJob]] completed with the following parameters: [{JobParameter{name='run.id', value=1, type=class java.lang.Long, identifying=true}}] and the following status: [COMPLETED] in 791ms

Job: [SimpleJob: [name=transaccionJob]] completed with the following parameters: [{JobParameter{name='run.id', value=1, type=class java.lang.Long, identifying=true}}] and the following status: [COMPLETED] in 1s234ms

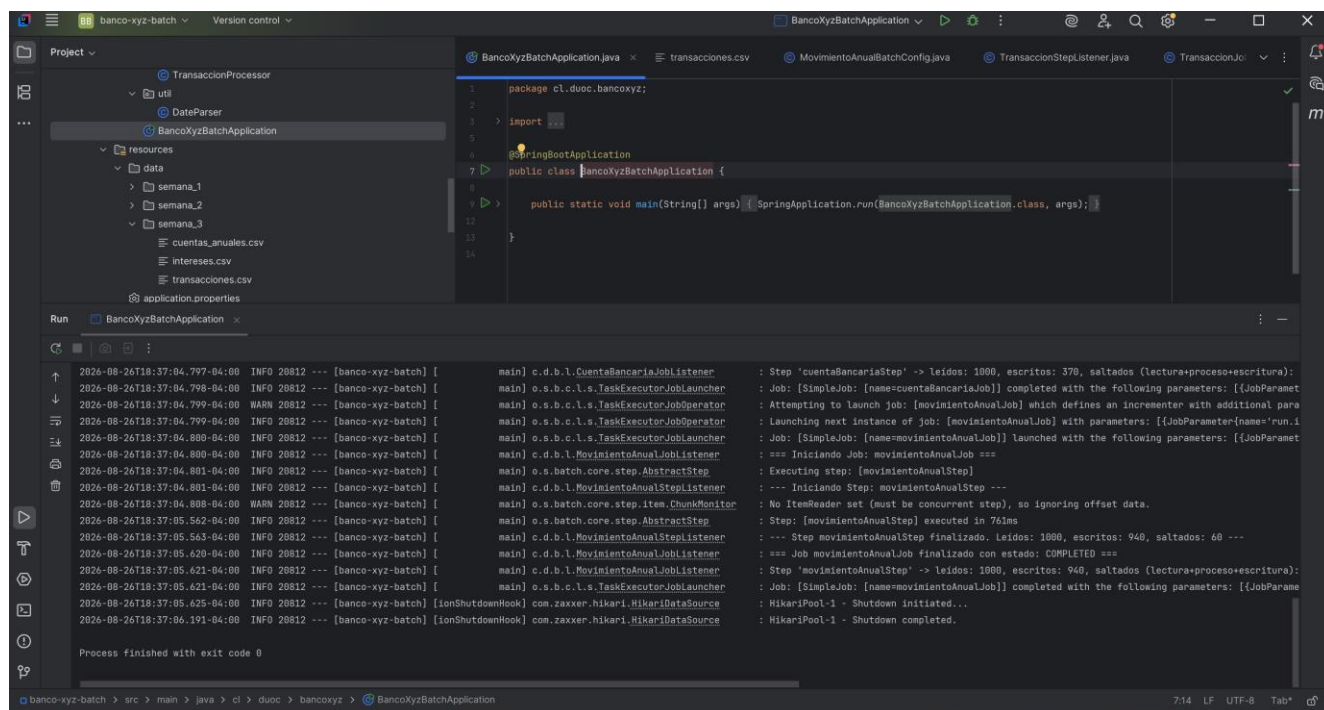
Hilos	Chunk	Tiempo Total(s)
6	500	2.781

Lógica de negocio por proceso:

- **Transacciones:** fecha válida (yyyy-MM-dd, yyyy/MM/dd, dd-MM-yyyy, dd/MM/yyyy); monto no vacío y mayor a cero (el tipo es medio de pago, no indica dirección del dinero, así que no afecta esta regla); tipo debe ser debito o credito.
- **Intereses:** nombre del titular no vacío ni "Unknown"; tipo de cuenta válido (ahorro, prestamo, hipoteca); edad y saldo razonables (edad 18-120, saldo no negativo); cálculo de interés y saldo final según tasa por tipo de cuenta.
- **Cuentas anuales:** tipo de transacción válido (deposito, retiro, compra, pago, normalizando depósito con tilde); monto no vacío y distinto de cero — se simplificó la regla original de signo según tipo de transacción (depósito positivo, retiro/compra negativo), dado el nivel de corrupción intencional del dataset en semana_3.

Evidencias

Ejecución y finalización proyecto Spring Batch



```
package cl.duoc.bancoxyz;

import org.springframework.boot.SpringApplication;

public class BancoXyzBatchApplication {

    public static void main(String[] args) {
        SpringApplication.run(BancoXyzBatchApplication.class, args);
    }
}
```

```
2024-08-26T18:37:04.797-04:00 INFO 20812 --- [banco-xyz-batch] [main] c.d.b.l.CuentaBancariaJobListener : Step 'cuentaBancariaStep' -> leídos: 1000, escritos: 370, saltados (lectura+proceso+escritura):
2024-08-26T18:37:04.798-04:00 INFO 20812 --- [banco-xyz-batch] [main] o.s.b.c.l.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=cuentaBancariaJob]] completed with the following parameters: [{JobParamet
2024-08-26T18:37:04.799-04:00 WARN 20812 --- [banco-xyz-batch] [main] o.s.b.c.l.s.TaskExecutorJobOperator : Attempting to launch job: [movimientoAnualJob] which defines an incremter with additional para
2024-08-26T18:37:04.800-04:00 INFO 20812 --- [banco-xyz-batch] [main] o.s.b.c.l.s.TaskExecutorJobOperator : Launching next instance of job: [movimientoAnualJob] with parameters: [{JobParameter{name='run.i
2024-08-26T18:37:04.800-04:00 INFO 20812 --- [banco-xyz-batch] [main] o.s.b.c.l.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=movimientoAnualJob]] launched with the following parameters: [{JobParamet
2024-08-26T18:37:04.801-04:00 INFO 20812 --- [banco-xyz-batch] [main] c.d.b.l.MovimientoAnualJobListener : === Iniciando Job: movimientoAnualJob ===
2024-08-26T18:37:04.801-04:00 INFO 20812 --- [banco-xyz-batch] [main] o.s.batch.core.step.AbstractStep : Executing step: [movimientoAnualStep]
2024-08-26T18:37:04.801-04:00 INFO 20812 --- [banco-xyz-batch] [main] c.d.b.l.MovimientoAnualStepListener : --- Iniciando Step: movimientoAnualStep ---
2024-08-26T18:37:04.808-04:00 WARN 20812 --- [banco-xyz-batch] [main] o.s.batch.core.step.Item.ChunkMonitor : No ItemReader set (must be concurrent step), so ignoring offset data.
2024-08-26T18:37:05.563-04:00 INFO 20812 --- [banco-xyz-batch] [main] o.s.batch.core.step.AbstractStep : Step: [movimientoAnualStep] executed in 761ms
2024-08-26T18:37:05.563-04:00 INFO 20812 --- [banco-xyz-batch] [main] c.d.b.l.MovimientoAnualStepListener : --- Step movimientoAnualStep finalizado. Leídos: 1000, escritos: 940, saltados: 60 ---
2024-08-26T18:37:05.620-04:00 INFO 20812 --- [banco-xyz-batch] [main] c.d.b.l.MovimientoAnualJobListener : === Job movimientoAnualJob finalizado con estado: COMPLETED ===
2024-08-26T18:37:05.621-04:00 INFO 20812 --- [banco-xyz-batch] [main] c.d.b.l.MovimientoAnualJobListener : Step 'movimientoAnualStep' -> leídos: 1000, escritos: 940, saltados (lectura+proceso+escritura):
2024-08-26T18:37:05.621-04:00 INFO 20812 --- [banco-xyz-batch] [main] o.s.b.c.l.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=movimientoAnualJob]] completed with the following parameters: [{JobParamet
2024-08-26T18:37:05.625-04:00 INFO 20812 --- [banco-xyz-batch] [main] o.s.b.c.l.s.TaskExecutorJobLauncher : HikariPool-1 - Shutdown initiated...
2024-08-26T18:37:06.191-04:00 INFO 20812 --- [banco-xyz-batch] [main] o.s.b.c.l.s.TaskExecutorJobLauncher : HikariPool-1 - Shutdown completed.

Process finished with exit code 0
```

Fragmento del log de consola al arrancar la aplicación. El warning "Asynchronous TaskExecutor detected with ItemStream reader" aparece una vez por cada uno de los 3

Jobs, confirmando que el TaskExecutor con 6 hilos fijos está correctamente configurado en cada Step. Es un aviso esperado y documentado de Spring Batch al combinar multithreading con un FlatFileItemReader (ver sección de "Limitación conocida" en el informe técnico), no un error de la aplicación.

```
: Starting BancoXyzBatchApplication using Java 25.0.2 with PID 14460 (C:\Users\jorge\OneDrive\Desktop\BancoXyzBatchApplication\BancoXyzBatchApplication.jar)
: No active profile set, falling back to 1 default profile: "default"
: Asynchronous TaskExecutor detected with ItemStream reader. This is probably an error, and may lead to unexpected behavior.
: DEPRECATION NOTE: The legacy implementation of the chunk-oriented processing model is deprecated and will be removed in a future version.
: Asynchronous TaskExecutor detected with ItemStream reader. This is probably an error, and may lead to unexpected behavior.
: DEPRECATION NOTE: The legacy implementation of the chunk-oriented processing model is deprecated and will be removed in a future version.
: Asynchronous TaskExecutor detected with ItemStream reader. This is probably an error, and may lead to unexpected behavior.
: DEPRECATION NOTE: The legacy implementation of the chunk-oriented processing model is deprecated and will be removed in a future version.
```

Tabla Transacciones

ID	registry	date	amount	type
1	141	2024-07-14	3000.00	debito
2	144	2024-12-30	3000.00	debito
3	147	2024-03-29	3000.00	credito
4	153	2024-07-28	1000.00	credito
5	156	2024-03-02	1500.00	debito
6	159	2024-09-01	1200.00	debito
7	162	2024-12-11	3000.00	debito
8	168	2024-10-24	1200.00	credito
9	177	2024-10-22	3000.00	debito
10	183	2024-01-21	3000.00	credito
11	192	2024-09-19	700.00	debito
12	195	2024-06-13	800.00	debito
13	198	2024-12-04	800.00	debito
14	204	2024-04-16	1200.00	debito
15	207	2024-02-08	3000.00	debito
16	210	2024-02-09	700.00	debito
17	213	2024-07-30	3000.00	debito
18	217	2024-02-21	1200.00	debito
19	240	2024-06-17	3000.00	debito
20	243	2024-10-17	800.00	debito
21	246	2024-02-10	1200.00	debito
22	252	2024-05-25	700.00	debito
23	261	2024-06-26	500.00	debito
24	276	2024-10-16	700.00	debito
25	303	2024-10-19	3000.00	credito
26	309	2024-03-23	1000.00	debito
27	323	2024-07-27	700.00	credito
28	337	2024-10-08	3000.00	debito
29	350	2024-02-02	1200.00	debito
30	366	2024-07-22	700.00	debito
31	367	2024-03-14	1500.00	debito
32	368	2024-07-26	800.00	debito
33	371	2024-12-10	1000.00	debito
34	386	2024-12-26	1200.00	debito

Tabla transacciones en PostgreSQL (Neon), tras la ejecución del transaccionJob sobre el dataset de semana_3 (1000 registros). Se omitieron 599 de los 1000 registros por la política de skip (errores de datos como monto vacío, negativo o cero, y tipo de transacción fuera de debito/credito; y fechas con formato no reconocido), cifra confirmada en la tabla execution_log. El DateParser extendido reconoce 4 formatos de fecha (yyyy-MM-dd,

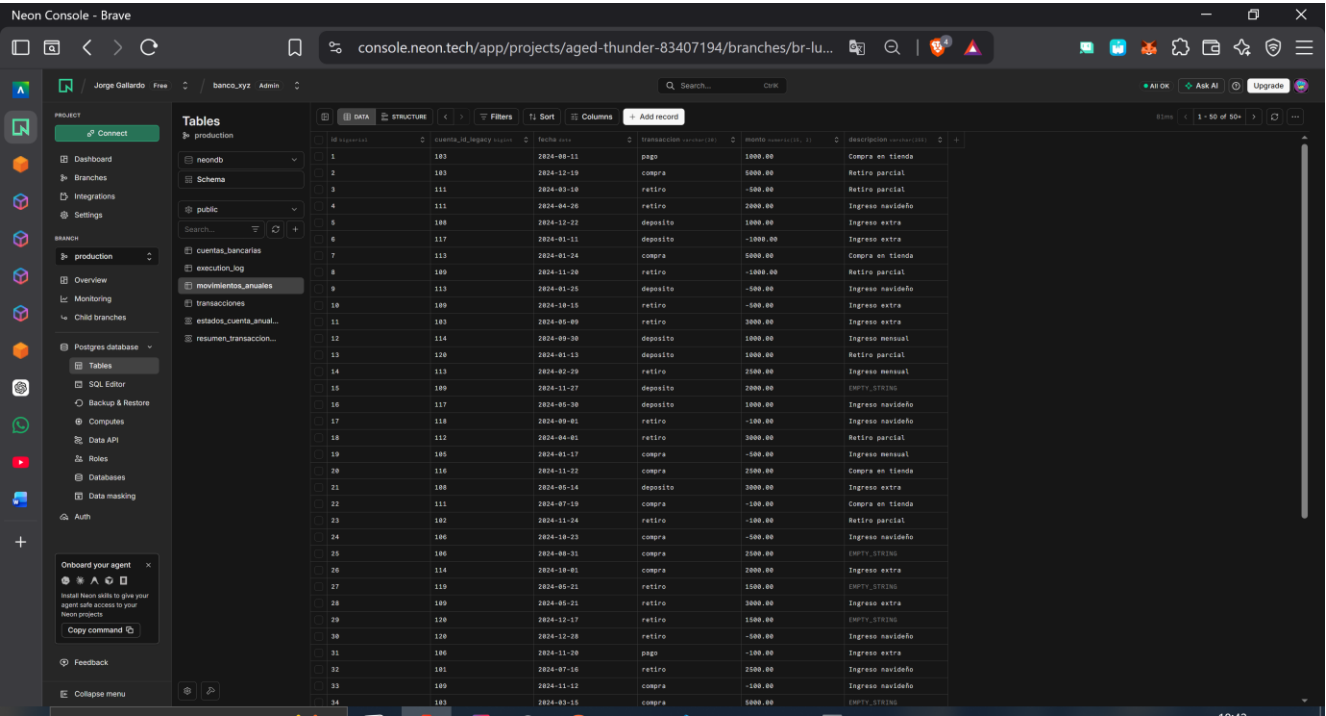
yyyy/MM/dd, dd-MM-yyyy, dd/MM/yyyy)), permitiendo procesar correctamente los 401 registros válidos.

--- Step transaccionStep finalizado. Leídos: 1000, escritos: 401, saltados: 599 ---

```
: Step: [transaccionStep] executed in 1s134ms
: --- Step transaccionStep finalizado. Leídos: 1000, escritos: 401, saltados: 599 ---
: === Job transaccionJob finalizado con estado: COMPLETED ===
```

id	job_nombre	step_nombre	estado	leídos	escritos	saltados	fecha_ejecucion
1	transaccionJob	transaccionStep	COMPLETED	1000	401	599	2026-08-26 22:37:03.84...

Tabla movimientos_anuales



id	cuenta_id_legacy	fecha	transaccion	monto	descripcion
1	103	2024-08-11	pago	1000.00	Compra en tienda
2	103	2024-12-19	compra	5000.00	Retiro parcial
3	111	2024-03-18	retiro	-500.00	Retiro parcial
4	111	2024-04-26	retiro	2000.00	Ingreso navideño
5	106	2024-12-22	deposito	1000.00	Ingreso extra
6	117	2024-01-11	deposito	-1000.00	Ingreso extra
7	113	2024-01-24	compra	5000.00	Compra en tienda
8	109	2024-11-28	retiro	-1000.00	Retiro parcial
9	113	2024-01-25	deposito	-500.00	Ingreso navideño
10	109	2024-10-15	retiro	-500.00	Ingreso extra
11	103	2024-05-09	retiro	3000.00	Ingreso extra
12	114	2024-09-30	deposito	1000.00	Ingreso mensual
13	120	2024-01-13	deposito	1000.00	Retiro parcial
14	113	2024-02-29	retiro	2100.00	Ingreso mensual
15	109	2024-11-27	deposito	2000.00	EMPTY_STRING
16	117	2024-05-30	deposito	1000.00	Ingreso navideño
17	118	2024-09-01	retiro	-100.00	Ingreso navideño
18	112	2024-04-01	retiro	3000.00	Retiro parcial
19	105	2024-01-17	compra	-100.00	Ingreso mensual
20	116	2024-11-22	compra	2100.00	Compra en tienda
21	109	2024-05-14	deposito	3000.00	Ingreso extra
22	111	2024-07-19	compra	-100.00	Compra en tienda
23	102	2024-11-04	retiro	-100.00	Retiro parcial
24	106	2024-10-23	compra	-600.00	Ingreso navideño
25	106	2024-08-01	compra	2100.00	EMPTY_STRING
26	114	2024-10-01	compra	2000.00	Ingreso extra
27	119	2024-05-21	retiro	1500.00	EMPTY_STRING
28	109	2024-06-21	retiro	3000.00	Ingreso extra
29	120	2024-12-17	retiro	1500.00	EMPTY_STRING
30	120	2024-12-28	retiro	-500.00	Ingreso navideño
31	106	2024-11-28	pago	-100.00	Ingreso extra
32	101	2024-07-16	retiro	2500.00	Ingreso navideño
33	109	2024-11-12	compra	-100.00	Ingreso navideño
34	103	2024-03-15	compra	5000.00	EMPTY_STRING

Tabla movimientos_anuales en PostgreSQL (Neon), tras la ejecución del movimientoAnualJob sobre el dataset de semana_3 (1000 registros). La validación de signo según tipo de transacción se simplificó a "monto distinto de cero", dado el nivel de corrupción intencional del dataset. Se omitieron 60 de los 1000 registros por la política de skip (montos vacíos o iguales a cero), cifra confirmada en la tabla execution_log.

```
: Step: [movimientoAnualStep] executed in 761ms
: --- Step movimientoAnualStep finalizado. Leídos: 1000, escritos: 940, saltados: 60 ---
: === Job movimientoAnualJob finalizado con estado: COMPLETED ===
```

	id	descripcion	descripcionStep	completo	total	progreso	fecha
☐	3	movimientoAnualJob	movimientoAnualStep	COMPLETED	1000	940	60

[illegible]

--- Step cuentaBancariaStep finalizado. Leídos: 1000, escritos: 370, saltados: 630 ---

```
: Step: [cuentaBancariaStep] executed in 773ms
: |-- Step cuentaBancariaStep finalizado. Leídos: 1000, escritos: 370, saltados: 630 ---
: === Job cuentaBancariaJob finalizado con estado: COMPLETED ===
```

2	cuentaBancariaJob	cuentaBancariaStep	COMPLETED	1000	370	630	2026-08-26 22:37:04.69...
3	movimientoAnualJob	movimientoAnualStep	COMPLETED	1000	940	60	2026-08-26 22:37:05.51...

Registros omitidos por skip

- **Transacciones:** 599 de 1000 registros omitidos — por errores de datos (monto vacío, negativo o cero; tipo fuera de debito/credito) y por fecha con formato no reconocido.
- **Intereses:** 630 de 1000 registros omitidos — por nombre vacío o "Unknown", saldo o edad vacíos, tipo de cuenta inválido, o edad fuera de rango.
- **Estados de cuenta anuales:** 60 de 1000 registros omitidos — por monto vacío o igual a cero.

En los tres casos, los registros inválidos se descartaron automáticamente gracias a BancoXyzSkipPolicy (la política de skip personalizada configurada en cada Step, con contadores independientes por tipo de excepción), sin detener la ejecución del resto del proceso. Las cifras están confirmadas en la tabla `execution_log`.

Resumen y auditoría (vistas SQL)

Se muestra la evidencia de las dos vistas de agregación, consultadas tras la ejecución de los Jobs sobre el dataset de semana_3.

```

66
67 DROP VIEW IF EXISTS resumen_transacciones_diarias;
68 CREATE OR REPLACE VIEW resumen_transacciones_diarias AS
69 SELECT fecha,
70        COUNT(*) AS cantidad_transacciones,
71        COALESCE(SUM(CASE WHEN tipo = 'debito' THEN monto ELSE 0 END), 0) AS total_debitos,
72        COALESCE(SUM(CASE WHEN tipo = 'credito' THEN monto ELSE 0 END), 0) AS total_creditos,
73        SUM(monto) AS monto_total
74 FROM transacciones
75 GROUP BY fecha
76 ORDER BY fecha;
77
78 DROP VIEW IF EXISTS estados_cuenta_anuales;
79 CREATE OR REPLACE VIEW estados_cuenta_anuales AS
80 SELECT cuenta_id_legacy AS cuenta_id,
81        COALESCE(SUM(CASE WHEN transaccion = 'deposito' THEN monto ELSE 0 END), 0) AS total_depositos,
82        COALESCE(SUM(CASE WHEN transaccion = 'retiro' THEN monto ELSE 0 END), 0) AS total_retiros,
83        COALESCE(SUM(CASE WHEN transaccion = 'compra' THEN monto ELSE 0 END), 0) AS total_compras,
84        SUM(monto) AS saldo_neto,
85        COUNT(*) AS cantidad_movimientos
86 FROM movimientos_anuales
87 GROUP BY cuenta_id_legacy
88 ORDER BY cuenta_id_legacy;

```

Vistas `resumen_transacciones_diarias` y `estados_cuenta_anuales` en PostgreSQL (Neon), consultadas tras la ejecución de `transaccionJob` y `movimientoAnualJob` sobre el dataset de `semana_3`.

Ambas vistas se definen en `schema-postgresql.sql`, el mismo script que crea las tablas del proyecto, y se recrean automáticamente cada vez que arranca la aplicación junto con las tablas, no como parte del código Java del batch.

Se implementan así por separación de responsabilidades: los recursos de cómputo del batch (hilos, chunks, memoria) están destinados a procesar y validar los datos de entrada, no a generar tablas de tipo informe de negocio, esta función se destina a la base de datos.

Incorporar ese calculo a un nuevo Step consumiría recursos de cómputo del proceso batch en una tarea que el motor de base de datos resuelve nativamente y sin costo adicional, ya que la vista se recalcula sola en cada consulta, sin duplicar datos ni requerir un Step extra.

The screenshot shows the Neon Console interface with a table view of the 'resumen_transaccion...' table. The table is grouped by 'fecha' (date) and displays the following columns: fecha, cantidad_transacciones, total_debitos, total_creditos, and monto_total. The data is organized into rows for each date from 2024-01-01 to 2024-02-27.

fecha	cantidad_transacciones	total_debitos	total_creditos	monto_total
2024-01-01	2	1200.00	800.00	2000.00
2024-01-02	1	0	700.00	700.00
2024-01-04	1	0	800.00	800.00
2024-01-06	1	3000.00	0	3000.00
2024-01-07	1	0	700.00	700.00
2024-01-09	1	0	800.00	800.00
2024-01-09	1	3000.00	0	3000.00
2024-01-12	1	1000.00	0	1000.00
2024-01-13	1	0	1000.00	1000.00
2024-01-14	2	800.00	800.00	1600.00
2024-01-15	3	700.00	1300.00	1300.00
2024-01-17	1	0	800.00	800.00
2024-01-19	3	1800.00	1000.00	2800.00
2024-01-21	4	3000.00	1000.00	4000.00
2024-01-22	3	800.00	2200.00	3000.00
2024-01-25	1	0	1000.00	1000.00
2024-01-26	1	0	1000.00	1000.00
2024-01-27	2	1200.00	1000.00	4200.00
2024-01-29	1	1500.00	0	1500.00
2024-01-30	1	0	800.00	800.00
2024-01-31	1	0	1000.00	1000.00
2024-02-02	2	1200.00	1000.00	4200.00
2024-02-03	4	3000.00	1000.00	5000.00
2024-02-09	3	2700.00	0	2700.00
2024-02-10	1	1200.00	0	1200.00
2024-02-11	1	1000.00	0	1000.00
2024-02-15	1	1000.00	0	1000.00
2024-02-16	2	700.00	1200.00	1300.00
2024-02-17	1	800.00	0	800.00
2024-02-19	1	0	1000.00	1000.00
2024-02-19	1	0	1000.00	1000.00
2024-02-20	2	1200.00	1000.00	4200.00
2024-02-21	2	1200.00	1000.00	4200.00
2024-02-24	1	800.00	0	800.00
2024-02-27	2	700.00	1000.00	1700.00

Vista resumen_transacciones_diarias en PostgreSQL (Neon), consultada tras la ejecución del transaccionJob sobre el dataset de semana_3. La vista agrega, por fecha, la cantidad de transacciones, el total de débitos, el total de créditos y el monto total, calculado directamente sobre la tabla transacciones mediante GROUP BY fecha. Al ser una vista (no una tabla poblada por el batch), se recalcula automáticamente en cada consulta, reflejando siempre el estado actual de los datos persistidos.

The screenshot shows the Neon Console interface with a table view of the 'estados_cuenta_anuales' table. The table displays the following columns: cuenta_id, total_depositos, total_retiros, total_compras, saldo Neto, and cantidad_movimientos. The data is organized into rows for each account ID from 101 to 120.

cuenta_id	total_depositos	total_retiros	total_compras	saldo Neto	cantidad_movimientos
101	36000.00	8200.00	11300.00	56900.00	43
102	22800.00	20500.00	21200.00	76600.00	51
103	8800.00	14500.00	41600.00	68900.00	39
104	14700.00	20200.00	26800.00	63200.00	51
105	23900.00	22800.00	14000.00	67800.00	56
106	26700.00	15000.00	35800.00	86900.00	58
107	24100.00	26800.00	20900.00	75000.00	54
108	13800.00	33700.00	22300.00	69200.00	51
109	39900.00	22400.00	12300.00	74100.00	47
110	18400.00	15100.00	15500.00	55800.00	44
111	38400.00	20900.00	10700.00	75000.00	44
112	27500.00	24300.00	19900.00	73100.00	46
113	28400.00	13400.00	10700.00	55500.00	37
114	21300.00	32800.00	30500.00	83300.00	54
115	14400.00	6700.00	25900.00	49900.00	41
116	39100.00	28400.00	23900.00	92900.00	55
117	24300.00	19800.00	16400.00	65000.00	37
118	38700.00	25900.00	28300.00	86400.00	49
119	23400.00	13400.00	34500.00	74300.00	46
120	19500.00	19400.00	15400.00	55300.00	37

Vista estados_cuenta_anuales en PostgreSQL (Neon), consultada tras la ejecución del movimientoAnualJob sobre el dataset de semana_3. La vista agrega, por cuenta, el total de depósitos, retiros, compras, el saldo neto y la cantidad de movimientos, calculado sobre la

tabla movimientos_anuales mediante GROUP BY cuenta_id_legacy. Cumple la función de informe de auditoría por cuenta solicitada en el enunciado, sin requerir un Step adicional dentro del Job de Spring Batch (ver justificación en la sección 9 del informe técnico).



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.